

Aula 17 – Neural ODEs e PINNs

Handout de Laboratório

Disciplina de Aprendizagem Profunda em Tempo Contínuo

Sumário

1	Objetivos da aula	1
2	Equações diferenciais e o modelo de Lotka–Volterra	1
2.1	Sistema Presa–Predador (Lotka–Volterra)	2
3	Neural ODEs e o pacote torchdiffeq	3
3.1	Neural ODE	3
3.2	odeint vs odeint_adjoint	3
4	PINNs e a equação do calor	4
4.1	Equação do Calor 1D	4
4.2	Ideia de PINN	4
4.3	Visualização da solução PINN	5
5	Resumo dos exercícios	5

1 Objetivos da aula

Nesta aula de laboratório vamos:

- Relembrar a formulação básica de equações diferenciais ordinárias (EDOs).
- Implementar um solver de Euler em NumPy para o sistema de Lotka–Volterra.
- Entender como o pacote torchdiffeq (odeint e odeint_adjoint) integra EDOs em modelos PyTorch.
- Comparar o custo de memória da autodiferenciação reversa padrão ($O(L)$) com o método adjunto ($O(1)$) em Neural ODEs.
- Implementar a ideia de *Physics-Informed Neural Networks* (PINNs) em JAX para a equação do calor.

O notebook associado (`aula16_neural_odes_pinns.ipynb`) contém duas versões: uma para estudantes (com trechos `TODO`) e uma versão gabarito.

2 Equações diferenciais e o modelo de Lotka–Volterra

Uma EDO escalar tem a forma

$$\frac{dh}{dt} = f(h, t), \quad h(0) = h_0.$$

No caso vetorial, $h(t) \in \mathbb{R}^d$ e temos

$$\frac{d\mathbf{h}}{dt} = \mathbf{f}(\mathbf{h}, t).$$

2.1 Sistema Presa–Predador (Lotka–Volterra)

O sistema de Lotka–Volterra é um modelo clássico de ecologia, com duas populações: presas (x) e predadores (y). A dinâmica é dada por:

$$\frac{dx}{dt} = \alpha x - \beta xy, \quad (1)$$

$$\frac{dy}{dt} = \delta xy - \gamma y, \quad (2)$$

onde:

- $\alpha > 0$: taxa de crescimento das presas na ausência de predadores;
- $\beta > 0$: taxa de encontro/consumo (efeito negativo dos predadores sobre as presas);
- $\delta > 0$: taxa de crescimento dos predadores ao consumir presas;
- $\gamma > 0$: taxa de morte dos predadores na ausência de presas.

No notebook, definimos:

- Uma versão **PyTorch** da dinâmica, como módulo `LotkaVolterra`, para uso com `torchdiffeq`.
- Uma versão **NumPy** da mesma dinâmica, `lotka_volterra_numpy`, para testar o método de Euler “na mão”.

Exercício 1.1 – Solver de Euler em NumPy

Dada a EDO

$$\frac{dh}{dt} = f(h, t), \quad h(t_0) = h_0,$$

o método de Euler avança a solução com passo Δt por:

$$h_{n+1} = h_n + \Delta t f(h_n, t_n),$$

onde $t_n = t_0 + n\Delta t$.

No notebook, a função

`numpy_euler_solver(func, h0, t_span)`

já está parcialmente escrita. Sua tarefa é:

- Calcular $f(h, t)$ chamando `func(h, t_curr)`.
- Atualizar h via $h \leftarrow h + dt \cdot f_val$.
- Armazenar a trajetória inteira em uma lista para visualização.

Exercício 1.2 – Explorar parâmetros do modelo LV

Na função

`lotka_volterra_numpy(h, t)`

you pode:

- Alterar $\alpha, \beta, \delta, \gamma$ e observar o efeito na dinâmica.
- Discutir em grupo como a dinâmica muda (explosão, extinção, ciclos mais amplos, etc.).

3 Neural ODEs e o pacote torchdiffeq

3.1 Neural ODE

Em uma Neural ODE, a dinâmica é parametrizada por uma rede neural:

$$\frac{d\mathbf{h}}{dt} = f_{\theta}(\mathbf{h}(t), t),$$

onde f_{θ} é tipicamente um MLP (como `DynamicsModel` no notebook). A solução no tempo final t_1 é obtida integrando numericamente:

$$\mathbf{h}(t_1) = \text{ODESolve}(f_{\theta}, \mathbf{h}_0, t_0, t_1).$$

O pacote `torchdiffeq` fornece a função

```
odeint(func, y0, t, method='dopri5')
```

que:

- Recebe uma função `func(t, y)` compatível com PyTorch.
- Integra a EDO para todos os tempos em `t`.
- Retorna um tensor de forma `[len(t), batch_size, dim]`.

3.2 odeint vs odeint_adjoint

Do ponto de vista de autodiferenciação:

- **Autodiff padrão (reverse mode):** Na prática, salva toda a trajetória interna do solver (todos os estados intermediários). Isso implica custo de memória proporcional ao número de passos de integração L : $O(L)$.
- **Método Adjunto:** Em vez de guardar a trajetória, reconstrói-a ao contrário na hora do backprop, resolvendo uma EDO adjunta. Isso reduz o custo de memória para $O(1)$ (em relação ao número de passos), às custas de mais tempo de computação.

Em `torchdiffeq`, isso se reflete em:

- `odeint`: autodiff padrão, consumo de memória maior.
- `odeint_adjoint`: usa o método adjunto, memória $O(1)$ em relação aos passos, ideal para solvers adaptativos em tempo contínuo.

Exercício 2.1 – Uso de `odeint` ($O(L)$)

Na função:

```
compare_autodiff(F_model, H0, T_span, true_output)
```

você deve:

- Usar `odeint(F_model, H0, T_span, method='dopri5')` para obter a trajetória completa.
- Selecionar o estado final: `H\FINAL\_STD = H\_TRAJ\_STD[-1]`.
- Calcular a loss `LOSS\_STD = mse(H\FINAL\_STD, true_output)`.
- Propagar gradientes (`LOSS_STD.backward()`) e registrar a norma dos gradientes.

Exercício 3.1 – Uso de `odeint_adjoint` (O(1))

Ainda em `compare_autodiff`, você deve:

- (a) Zerar gradientes do modelo.
- (b) Usar `odeint_adjoint(F_model, H0, T_span, method='dopri5')`.
- (c) Obter o estado final `H_FINAL_ADJ = H_TRAJ_ADJ[-1]`.
- (d) Calcular `LOSS_ADJ` e `.backward()`.
- (e) Comparar `grad_norm_std` e `grad_norm_adj`, bem como o tempo e consumo de memória (discussão qualitativa).

4 PINNs e a equação do calor

4.1 Equação do Calor 1D

Consideramos a EDP:

$$u_t(x, t) = \nu u_{xx}(x, t),$$

onde:

- $u(x, t)$ é a temperatura em posição x e tempo t ;
- $\nu > 0$ é o coeficiente de difusão térmica.

Usaremos uma condição inicial do tipo:

$$u(x, 0) = \sin(\pi x), \quad x \in [-1, 1].$$

A solução analítica, neste caso, é:

$$u(x, t) = e^{-(\pi^2 \nu)t} \sin(\pi x).$$

4.2 Ideia de PINN

Uma *Physics-Informed Neural Network* aproxima $u(x, t)$ com uma rede neural $u_\theta(x, t)$. A função de perda típica combina dois termos:

- **Perda de dados/fronteira:** força a rede a satisfazer condições iniciais e de contorno:

$$\mathcal{L}_{\text{boundary}} = \frac{1}{N_b} \sum_{i=1}^{N_b} \left(u_\theta(x_i^b, t_i^b) - y_i^b \right)^2.$$

- **Perda física (resíduo da EDP):** força a rede a satisfazer a equação do calor em pontos de colação (x_j, t_j) :

$$r_\theta(x, t) = u_t(x, t) - \nu u_{xx}(x, t), \quad \mathcal{L}_{\text{física}} = \frac{1}{N_f} \sum_{j=1}^{N_f} r_\theta(x_j, t_j)^2.$$

A loss total é:

$$\mathcal{L}(\theta) = \mathcal{L}_{\text{boundary}} + \mathcal{L}_{\text{física}}.$$

Exercício 4.1 – Interpretar a loss PINN no código

No notebook, a função `pinn_loss` já implementa:

- Cálculo da loss de fronteira (condição inicial) usando `boundary_x`, `boundary_t`, `boundary_y`.
- Cálculo da loss física chamando `pde_residual` em pontos do domínio.

Tarefa: Relacionar cada termo da loss com as fórmulas da seção anterior.

Exercício 4.2 – Implementar o resíduo da EDP

Na função:

```
pde_residual(params, x, t, nu)
```

you deve:

- (a) Definir $u_\theta(x, t)$ via `mlp_forward(params, jnp.array([x, t]))`.
- (b) Calcular $u_t(x, t)$ usando `grad` em relação a t .
- (c) Calcular $u_{xx}(x, t)$ usando `grad(grad(...))` em relação a x .
- (d) Retornar o resíduo $u_t - \nu u_{xx}$.

4.3 Visualização da solução PINN

Após o treinamento, é interessante visualizar $u_\theta(x, t)$ em todo o domínio e compará-lo com a solução analítica. No notebook, adicionamos uma função de visualização que:

- Gera um *heatmap* de $u_\theta(x, t)$ no plano (x, t) .
- Compara cortes em tempo fixo t^* com a curva analítica.

5 Resumo dos exercícios

- **Exercício 1.1:** Completar o solver de Euler em NumPy.
- **Exercício 1.2:** Explorar parâmetros de Lotka–Volterra.
- **Exercício 2.1:** Usar `odeint` e analisar gradientes ($O(L)$).
- **Exercício 3.1:** Usar `odeint_adjoint` e discutir $O(1)$ memória.
- **Exercício 4.1:** Interpretar a loss de PINN.
- **Exercício 4.2:** Implementar u_t , u_{xx} e o resíduo da equação do calor.